Besides that, one can use `deep_stringify_keys` and `deep_stringify_keys!` to stringify all the keys in the given hash and all the hashes nested into it. An example of the result is:

```
{nil => nil, 1 => 1, nested: {a: 3, 5 => 5}}.deep_stringify_keys
# => {""=>nil, "1"=>1, "nested"=>{"a"=>3, "5"=>5}}
```

Defined in `active_support/core_ext/hash/keys.rb`.

11.4.4 `symbolize_keys` and `symbolize_keys!`

The method `symbolize_keys` returns a hash that has a symbolized version of the keys in the receiver, where possible. It does so by sending `to_sym` to them:

```
{nil => nil, 1 => 1, "a" => "a"}.symbolize_keys
# => {1=>1, nil=>nil, :a=>"a"}
```

Note in the previous example only one key was symbolized.

In case of key collision, one of the values will be chosen. The chosen value may not always be the same given the same hash:

```
{"a" => 1, a: 2}.symbolize_keys
# The result could either be
# => {:a=>2}
# or
# => {:a=>1}
```

This method may be useful for example to easily accept both symbols and strings as options. For instance `ActionController::UrlRewriter` defines

```
def rewrite_path(options)
  options = options.symbolize_keys
  options.update(options[:params].symbolize_keys) if options[:params]
  ...
end
```

The second line can safely access the `:params` key, and let the user to pass either `:params` or "params".

There's also the bang variant `symbolize_keys!` that symbolizes keys in the very receiver.

Besides that, one can use `deep_symbolize_keys` and `deep_symbolize_keys!` to symbolize all the keys in the given hash and all the hashes nested into it. An example of the result is:

```
{nil => nil, 1 => 1, "nested" => {"a" => 3, 5 => 5}}.deep_symbolize_keys
# => {nil=>nil, 1=>1, nested:{a:3, 5=>5}}
```

Defined in `active_support/core_ext/hash/keys.rb`.

11.4.5 `to_options` and `to_options!`

The methods `to_options` and `to_options!` are respectively aliases of `symbolize_keys` and `symbolize_keys!`.

Defined in `active_support/core_ext/hash/keys.rb`.

11.4.6 `assert_valid_keys`

The method `assert_valid_keys` receives an arbitrary number of arguments, and checks whether the receiver has any key outside that white list. If it does `ArgumentError` is raised.

```
{a: 1}.assert_valid_keys(:a)  # passes
{a: 1}.assert_valid_keys("a") # ArgumentError
```

Active Record does not accept unknown options when building associations, for example. It implements that control via `assert_valid_keys`.

Defined in `active_support/core_ext/hash/keys.rb`.

11.5 Working with Values

11.5.1 `transform_values` && `transform_values!`

The method `transform_values` accepts a block and returns a hash that has applied the block operations to each of the values in the receiver.